To what extent does incorporating gameful design principles into a playful learning environment enhance Level 3 students' perceptions of their own computational thinking skills?

Robert Lancaster
Games Academy
Falmouth University
Falmouth, England
RL272978@falmouth.ac.uk

***Abstract*—In recent years, games have been increasingly utilised as effective learning environments. Many of these educational environments take the form of 'serious games', whereas other games put a greater emphasis on creativity and imagination through playful learning, in which valuable skills can be learned in the process of play, often without identification or recognition. In this paper, the extent to which the self-perception of playfully learned computational thinking (CT) skills can be enhanced was investigated, achieved through the incorporating the gameful design principle of 'implicit progression' into a game-based learning environment, in which progressional rewards implicit to the process of problem-solving were used to identify the implementations of CT concepts performed by student participants. Results showed a significant increase in the confidence students had to provide a self-assessment of their CT skills as well as a significantly increased level of self-efficacy, with no significant impact on student interest in future problem-solving. However, due to low statistical power and participant limitations, it was concluded that further research is required to understand if the application of gameful design principles has a significant effect on the self-perception of CT skills, with recommendations for future work to investigate additional principles of gameful design.**

***Index Terms*—Games, playful learning, gameful design**

I. Introduction

Games have been used in educational contexts for many years, beginning with games such as 'The Sumerian Game' in 1964 [1] or 'The Oregon Trail' in 1971 [2]. Similar to their physical counterparts, this is due to the ability games have to motivate and engage players [3] with elements of interaction, immediate feedback and self-direction [4]. Within these experiences, learning becomes intrinsic [5], inherent to the process of fulfilling the goals of the game, and is consequently a subsidiary to the elicited motivators of the game [6]. In the context of eliciting creativity, this implicit process [7] follows the approach of playful learning [8] [9].

It is possible to learn a variety of skills through playful learning [10], as long as those skills can be explored through a creative and imaginative approach. One such example, and the focus of this paper, is computational thinking (CT) skills [11] [12], relating to the thought processes and problem-solving methods useful for any field involving complex problems [13], including computer science and game development.

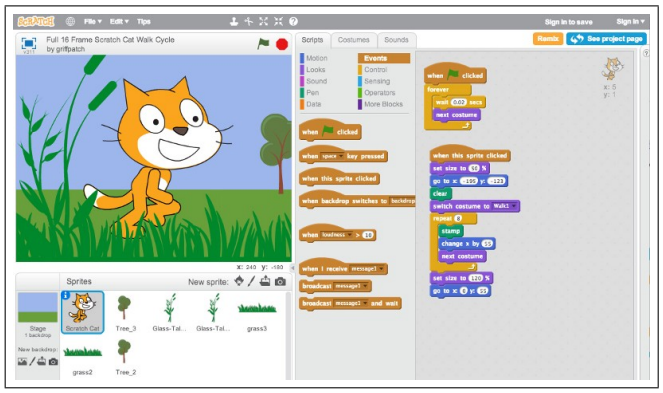


Figure 1. Scratch [14], a visual programming language and game creation platform teaching computational thinking skills through playful learning [15].

A. Computational Thinking in Game-based Learning

Playful learning within digital games, also known as Game-based Learning (GBL) [16], is used within a large number of platforms for a variety of disciplines, such as 'Kahoot!' [17], a quiz learning platform, 'Duolingo' [18], a language learning app, as well as conventional video games like Kerbal Space Program [19] for learning physics through simulation, Notably however, GBL platforms for CT skills often take the specific form of game-creation platforms [20].

These platforms, including examples such as Scratch [14], Roblox [21] and Kodu Game Lab [22], gives users an accessible interface to program the logic and behaviours necessary for their own playable games, encouraging the playful practice of problem-solving characteristic of CT skills. These experiences are often introductory for wider technical and problem-solving disciplines, such as programming and computer science [23], in which players who enjoy playing games are intrinsically motivated to learn how to create their own games. This points to the effectiveness of GBL environments for teaching CT skills, with additional justifications for this link explored further in this paper. It is certain, however, that the development of CT skills and effective methods for teaching them is a need [24]–[26] that software is developed to meet.

B. Recognition of Implicitly Learnt Skills

Though it has been shown that CT skills can be learned in the process of play, how this method of education affects the recognition and identification of these skills is unclear. The practice-based education and application of CT skills for creating solutions to problems, characteristic of implicit playful learning, does not involve the identification of the skills learned and used to create those solutions.

In that sense, unlike the structured and declarative nature of explicit learning [27], there is no proven link between skills learned in the process of play and their conscious recognition. Playful learning may therefore be limited in its capacity to build the self-perception [28] students have of their own CT skills as they learn and apply them through play.

C. Importance of Self-perception

When engaging with any field of study, it is incredibly important to have an accurate perception of your own capabilities to establish the basis for future learning and instil a sense of confidence in the field [29]. This self-perception also affects decision-making for future study and industry prospects [30], which is especially important for younger students who are deciding the disciplines they want to pursue in life.

Similarly, it is younger students who benefit the most from playful learning [31], with its explorative and creative characteristics facilitating their natural curiosity [32]. This also aligns with the needs of the wider industries relevant to CT skills, with reports [33] [34] indicating that the UK requires a greater focus on school education [35] to better support the declining growth of its computing industries.

Therefore, this paper aims to investigate how the recognition of implicitly learned CT skills can be improved during the process of play, with the aim of better supporting the continued development of CT skills initiated by playful learning, consequent progression towards relevant industries and overall improvement of self-efficacy for further learning.

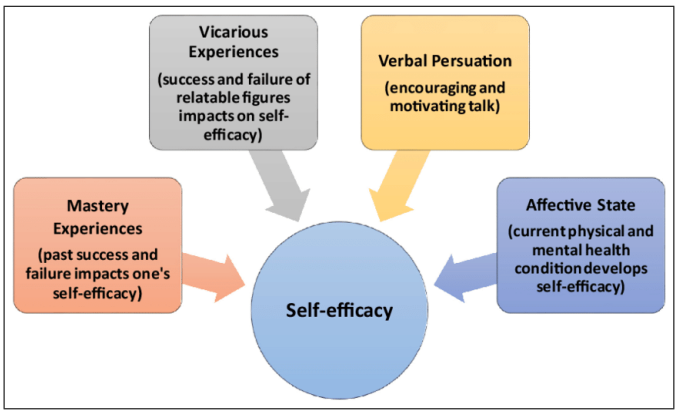


Figure 2. Self-efficacy [36] is a subset of self-perception, relating to the belief or confidence an individual has in their own capabilities. High self-efficacy is correlated with promoting perseverance and accomplishment.

D. Enhancing Self-perception

A consideration for developing solutions for enhanced self-perception is that the same intrinsic qualities that obscure the perception of learnt skills also contribute to the effectiveness of playful learning, in which the lack of awareness of what is being learned allows for reduced cognitive demand, subsequently prioritising intrinsic motivation and thus increasing the ease of engagement [37]. Compromising a playful approach that maintains the intrinsically motivated implicit learning of CT skills with the enhanced recognition of those skills is consequently a challenge for this research to overcome.

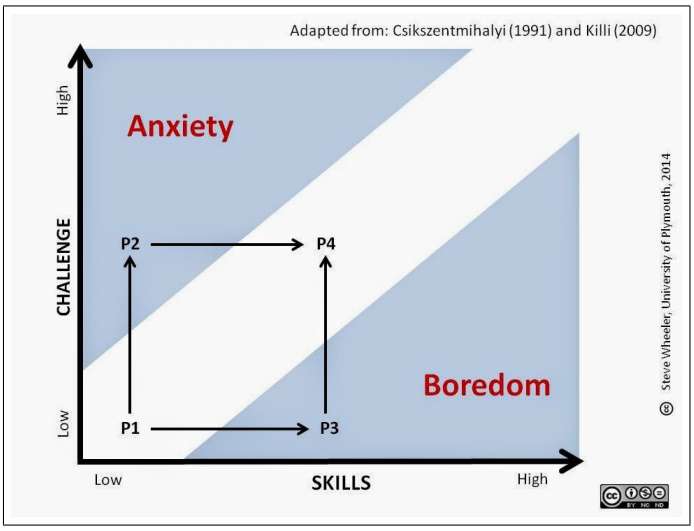


Figure 3. Flow theory [38] describes the state of deep immersion, focus and intrinsic motivation [39] contributing to an optimal learning environment.

It follows that the solution should not interrupt the 'flow state' [40] of heightened implicit learning [41] core to playful education. To this extent, the principles of 'Gameful Design' [42] will be researched, a concept sharing similar values of learning flow and intrinsic motivation.

This paper plans to incorporate the specific gameful design element and individual motivator of progression [43] [44] into a playful learning environment, established implicitly by embedding rewards and measurable progress within the learning process. These progressive rewards will support the CT skills used during the process of play, with the goal of maintaining intrinsic motivation as the primary motivator.

Following this, the rewards of implicit progression will be used connect the playful learning process with the actual CT skills being learned and applied, such that students will be able to recognise the skills they are using to create their solution to a problem and thus better inform their self-perception. The progression of these rewards is also intended to signal their learning progress [45] without explicitly defining it.

Through this, the application of CT skills will be simultaneously identifiable and playful in their use, recognised as 'tools' for creation with their creative use for solving a problem remaining the inherent reward for learning those skills.

II. Background

A. Playful Learning

The book 'Play' [46] by Catherine Garvey defines the activity of play by five elements.

1) Positively valued by the player.
2) Self-motivated and lacking extrinsic goals.
3) Freely chosen and spontaneous in nature.
4) Actively engaging.
5) Possessing systematic relations to what is not play.

Consequently, learning through play (playful learning) is the process of acquiring skills through an activity or environment that possesses these qualities. As stated by [9], the benefits of this include developing or promoting creativity, imagination and spontaneous learning with a Constructivist approach [47].

1) Applications in Education: Playful learning is often applied to teaching young children, allowing them to test out ideas, explore new possibilities and refine their creative instincts [8]. However, many sources [8], [9], [31], [48]–[50] also indicate the usefulness and need to apply this to education for teenagers and adults, owing to the cultivated willingness to try something new and attempt something difficult where success is not guaranteed [48], in which the learning process is supported by failure [49] and thus differs from the fear of failing, avoidance of risk and extrinsic goal-orientation characteristic of higher education [50].

Implicit structures are inherently present in playful learning, identified by its qualities of openness, acceptance of risk-taking and failure, and intrinsic motivation [51]. As a result, it easily supports implicit learning, defined by learning without conscious awareness or intention [7]. A further variety of well-studied educational benefits are associated with implicit learning, including the immersive and optimal state of 'flow' [38]–[41], improved skill retention over time [52] and the enhancement of intuitive decision-making [53].

2) Game-based Learning: Video games as an activity have the same aforementioned elements of play [46] intentionally integrated in their design, and can thus be used as playful learning environments. This is known as Game-based Learning (GBL) [16], where the specific principles and characteristics of games [54]–[57] are used to create effective learning activities.

The advantages and disadvantages of GBL are explored in [3], noting its positive effect on the following:

- Student Motivation and Engagement
- Teamwork
- Quick Feedback and Progress Record
- Creativity and Lateral Thinking
- Risk-taking and Experimentation
- Preparation for Future Jobs

The paper also mentions the disadvantages in hindering physical play and high equipment costs, though concludes overall that the advantages of adding GBL to the classroom far outweighs its disadvantages. This is further supported by additional studies examining the merits of GBL for academic learning [58], [59], agreeing upon advantages of social engagement and encouraged creativity, with a focus on the game 'Minecraft' [60], a voxel-based sandbox game with an Education edition [61] developed for game-based learning.

B. Computational Thinking Skills

Computational thinking (CT) [11] [62] is defined as the ability to understand a complex problem and develop possible solutions that can be presented in a way that a computer or human can understand.

The four cornerstones of CT [11] are defined as such:

- **Decomposition** - breaking down a complex problem or system into smaller, more manageable parts.
- **Pattern Recognition** - looking for similarities among and within problems.
- **Abstraction** - focusing on the important information only, ignoring irrelevant details.
- **Algorithms** - developing a step-by-step solution to the problem, or the rules to follow to solve the problem.

Jeannette Wing's article [63] puts forward that CT skills, though typically associated with computer science fields, can be applied to any field that involves problem-solving, justified by the following characteristics:

1) Conceptualising, not programming
2) Fundamental, not a rote skill
3) A way that humans, not computers, think
4) Combines mathematical and engineering thinking
5) Ideas, not artifacts
6) For everyone, everywhere

A review of CT studies [64] finds that CT skills are primarily taught through the lens of computer science, including program design and visual programming languages. The review [64] also shows that the learning strategies of Problem-based Learning (PBL) [65], Cooperative Learning (CL) [66] and the previously discussed Game-based Learning (GBL) [16] are the most common strategies used to teach CT skills.

1) Problem-based Learning: PBL [67] is described by as a pedagogical approach that enables students to learn by engaging with meaningful problems, thus serving as a way to learn and improve problem-solving skills like computational thinking in a constructive, self-directed and contextual activity.

Its facilitation of learning CT skills [68] can come in many forms, though most often within the field of computer science [64]. This is because both the software engineering [69] and hardware architecture [70] of computers utilise the algorithmic logic core to computational thinking, subsequently solving or including problems that can be solved through decomposition, pattern recognition and abstraction.

To this extent, the concepts of computer science act as a mirror to the skills of computational thinking [71], in which the instrument of programming or process of designing hardware systems in PBL require students to exercise the same logical thought processes needed for a working solution, and verifiably illustrates the logical result of those thought processes in a way that is tangible and reproducible through the execution of a developed solution.

This allows students to reflect on their understanding of the problem, dynamically adjust their thinking and recognition of patterns, as well as test and experiment with small alterations, leading to further improvement of their CT skills through its iterative application in active problem solving.

C. Self-perception

Including concepts of 'self-efficacy' or 'academic self-concept' (ASC) [72] in the context of academia, self-perception of skills is described by the awareness someone has of their own abilities [73], and their consequent understanding of how their abilities can be used to achieve a particular goal.

It has strong relations to Self-perception theory, defined by Daryl Bem [74] as the development of attitudes by self-observation of behaviour and interpreting the attitudes that caused it. Following this, the self-observation of skills that a student learns or exercises is a part of developing their self-perception and self-concept relevant to the field of those skills.

The development of self-concept, especially ASC, is a well-studied yet contended field, discussing a variety of approaches including improved peer feedback [75] and mastery goals [76], as well as study into its effects on academic achievement and motivation [77]–[79]. This will be examined further in the contextual analysis, with specific reference to CT skills.

D. Gameful Design and Implicit Progression

As laid out in the study by S.Deterding [80], Gameful Design differs from the concept of Gamification:

- **Gamification** is the use of game design elements used to engage players in non-game contexts.
- **Gameful Design** is designing for gamefulness, the experiential quality of playing games [81], using game design thinking [42], [82].

Following this, gamification is characterised by explicit goals that stimulate extrinsic motivation [83], such as achievements, progress bars and points [84], though can consequently invite intrinsic motivation through the inherent enjoyment and self-motivated interaction with those elements. Gameful design, however, only aims to motivate intrinsically [85] by supporting a personalised experience of player autonomy, defined by the individual and inherent goals of the player and motivated through a playful experience.

1) Intrinsic Motivation: Intrinsic motivation, and gameful design by proxy, has roots in Self-determination Theory (SDT) [86], a complex field of research that connects the ideas of autonomy, competence and relatedness as the psychological needs that underpin motivation [87] [88]. It puts forward that motivation exists on a continuum from intrinsic to extrinsic, and by fostering autonomy, competence and relatedness in an activity, intrinsic motivation is stimulated.

This is the goal of gameful design, and by extension games in general, with different elements of game design intended to promote different or multiple aspects of SDT, consequently engaging a player with any kind of applicable activity. With this in mind, a specific element of gameful design relevant to this study is progression.

2) Implicit Progression: As described in [89], game progression is how the aspects of a game work together to provide players a sense of forward momentum and achievement, attained through a cycle of completing goals with rewarding outcomes. The overall result of this is increased played retention [90]. It follows that implicit progression is the implementation of this cycle as an inherent and unconscious part of gameplay [91], as opposed to an explicit primary goal.

III. Contextual Analysis

A. Distinguishing Implicit and Explicit Learning

Rod Ellis states two principle ways cognitive psychologists distinguish implicit and explicit learning [92]:

1) **Subsymbolic knowledge** vs **Symbolic knowledge**
   Implicitly learnt knowledge is subsymbolic, reflecting sensitivity to the structure of learned material, whereas explicitly learnt knowledge is symbolic, involving memorization with heavy demands on working memory
2) **Unawareness of learning** vs **Awareness of learning**
   Implicit learning leads to an unawareness of the learning that has taken place despite its evidence in behaviour, and the inability to verbalize what has been learned, whereas awareness of learning remains and can be verbalized in explicit learning.

This lack of learning awareness and inability to verbalize implicitly learnt knowledge, though well-supported and agreed upon by many other sources of research within the field of psychology [27], [93]–[96], is notably not well studied in its association with the development of self-efficacy, nor how the self-perception of implicitly learnt skills can be enhanced.

Notably however, a positive effect of explicit learning for the development of self-efficacy can be found in many studies [97]–[99], also corroborated within the discipline of learning CT skills for programming [100]. This may draw attention to the merits of transforming implicit CT knowledge into explicit knowledge to enhance self-perception of CT skills.

B. Computational Thinking Self-efficacy

In a study researching student CT self-efficacy [101], Luzia Leifheit investigates how student motivation and self-concept can be reliably assessed within early computing education, as well as how CT can be taught in a way that is motivating and beneficial for student self-concept. The paper asserts that positive self-concept is an important predictor of achievement, similarly supported by aforementioned studies [75] [76]. Leifheit's study involves the development of a questionnaire assessing self-concept and attitude towards programming [102], as well as a course teaching the essential CT concepts to young students, taught through embodied learning [103], game-based learning (GBL), scaffolding [104] and blended learning [105].

The study [101] consequently finds a significant positive effect of their developed teaching course on the students' experience and understanding of programming, programming-related self-concept and intrinsic value belief about programming, stating that GBL methods were generally suitable for teaching CT to young students, noting that conditional branching and nested conditionals were particularly challenging to teach.

Further research on student self-perception with regards to CT skills is notably sparse, with only a few studies investigating this particular relationship. In one such study [106], the effect algorithmic education has on student self-efficacy perception and CT skills, taught through an unplugged and non-gameful approach, is concluded to be effective for increasing self-efficacy and skills in the sub-dimension of algorithmic thinking, but to have no effect on the self-efficacy or proficiency in CT skills as a whole.

Another paper researching programming self-efficacy [107] identifies that students with higher programming self-efficacy study more and get higher grades, whereas those with a belief in fixed programming aptitude and a low level of programming self-efficacy tend to have more course fails and thus repeats. This further supports the importance of positive self-perception for academic success as put forward by Leifheit's paper [101], with an added suggestion for more incremental steps and improved feedback embedded in study to better support mastery experiences [76] that build strong self-efficacy.

To this extent, all papers [101], [106], [107] illustrated the greater need for improved student self-efficacy in CT skills, but lacked deeper insight into how student self-perceptions resultant from playful CT learning processes can be enhanced. Notably however, the recommendation of [107] may reflect the benefits of incorporating implicit progression for improving self-perception, in which progressive rewards act as incremental mastery experiences during the learning process.

C. Game-based Learning for Computational Thinking

A recent study [108] exploring the educational trends and applications of integrating CT and game-based learning (GBL) with design thinking [109] finds a mainstream research trend in targeting K-12 students, linked with an importance of CT in K-12 education. The study indicates that visual programming based on Scratch [15] is the main learning and development tool used in CT education, with interdisciplinary strategies of educational robotics, creativity and critical thinking being used in recent game design studies on CT.

The study [108] also finds that recent research trends have moved beyond simple cognitive evaluation of CT, with affective considerations of self-efficacy and attitude as the most used measures for assessing engagement in game-making activities. A recommendation is made for future research in CT learning to further investigate the impact of affective aspects and differences in cognitive levels, with further exploration of the affective effects of prior knowledge/experience, sex and national or cultural differences.

Lastly, the study [108] draws attention to how the elements of CT, design thinking (DT) and game design complement each other to foster learner's creativity, problem-solving abilities and hands-on skills, supported by a constructionism theory [47], in which CT provides the logical and structural framework, DT introduces creativity and improvements in user experience, and the hands-on activity of game design allows learners to develop new knowledge and deepen their conceptual understanding. Incorporating these elements together is concluded to address issues of equity, accessibility and scalability, as well provide skills for game-creation through the combination of logical and creative problem-solving in CT and DT, resulting in learners creating higher-quality games.

The benefits of GBL for learning CT skills is further investigated Yanjun Pan's paper [110], which finds that the use of Minecraft [60] resulted in a significant increase in students' CT self-efficacy, though did not result in a significant impact on CT competency or game-based engagement.

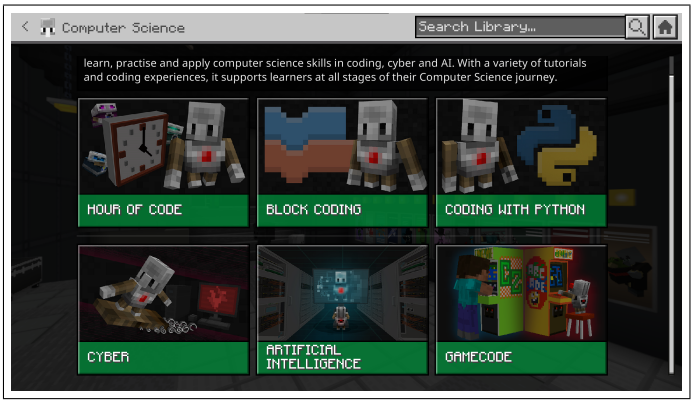


Figure 4. Minecraft Education [61] contains a variety of educational challenges, though as shown in [110], the standard edition of Minecraft [60] can be modified and similarly used for game-based learning.

Y. Pan [110] makes recommendations to investigate the in-game actions of students to better understand how the learning processes of CT competency can be enhanced during GBL, as well as proposes researchers to design learning support features aimed at assisting the transformation of students' learned implicit knowledge from GBL into explicit knowledge.

In another example of Minecraft being used for developing students' CT skills [111], Emine Kutay conceptualizes CT skills to encapsulate the use of CT practices, including testing and debugging, alongside knowledge of CT concepts. In the study, the role of Minecraft-based coding activities was found to be significant in its improvement of students' CT concept knowledge and practices, in which almost all students successfully used sequences, events and loops, with most struggling to use conditionals, operators and variables. E.Kutay altogether considers Minecraft an effective environment for learning simplistic CT concepts such as sequences, events and loops, as well as potentially complex computational concepts such as loops, though does not measure student self-efficacy relative to CT concepts or practices learnt through GBL in Minecraft.

Although both Y.Pan [110] and E.Kutay [111] involve middle school participants, their results are somewhat in contrast to one another, in which the former [110] concludes no significant impact on CT competency whereas the latter study [111] shows a significant improvement of CT skills. However, both studies [110] [111] agree on the merits of leveraging GBL as an alternative to traditional classroom teaching activities for computational thinking. Additional studies corroborate on the use of Minecraft as an effective environment for GBL [58], [59], [112]–[114], with one such study [115] concluding Minecraft to share many identifying features of real-world engineering disciplines through its Redstone mechanics.

D. Identified Research Gap

Altogether, though all analysed studies displayed strong support for enhancing CT self-perception, as well as the effectiveness of GBL for learning CT skills, there is a clear gap in existing research for how the self-perception of implicitly learnt CT skills through GBL can be enhanced, with one study [110] recommending further research into better supporting the transformation of implicit knowledge into explicit knowledge.

Table I
HYPOTHESES TABLE.

	Hypothesis	Null Hypothesis	Data Source
H1	Students engaging with a playful learning environment incorporating implicit progression will provide a higher level of confidence in their assessment of their CT skills compared to students using an unmodified version of the environment.	There is no significant increase in the confidence of self-assessment of students who engage with a playful learning environment incorporating implicit progression compared to students using an unmodified version of the environment.	Questionnaire
H2	Students engaging with a playful learning environment incorporating implicit progression will provide a higher rating of proficiency in their assessment of their CT skills compared to students using an unmodified version of the environment.	There is no significant increase in the self-assessed rating of proficiency of students who engage with a playful learning environment incorporating implicit progression compared to students using an unmodified version of the environment.	Questionnaire
H3	Students engaging with a playful learning environment incorporating implicit progression will express greater interest in further problem-solving involving CT skills compared to students using an unmodified version of the environment.	There is no significant increase of interest in further problem solving of students who engage with a playful learning environment incorporating implicit progression compared to students using an unmodified version of the environment.	Questionnaire

IV. RESEARCH QUESTION & HYPOTHESES

The research question is: Does incorporating the gameful design principle of implicit progression into a playful learning environment significantly enhance the self-assessment students have of the computational thinking skills they use to solve a problem within that environment?

The hypotheses to address this question are outlined above in Table I, in which the primary focus of this study is whether incorporating implicit progression causes the confidence of assessment to increase as examined by H1. This reflects the ability a student has to formulate a self-assessment of their CT skills, in which a higher confidence to do so is to be interpreted as an enhancement of self-perception.

However, H2 and H3 are important considerations for examining the impact on academic confidence, self-efficacy and attitude towards future-learning as important benefits and predictors of an improved self-perception of skills. This research provides no consideration for the accuracy of self-assessments, as enhancing the accuracy of self-perception is irrelevant to the research goals of the study.

V. METHODOLOGY

To address the research question and hypotheses, an **A/B** test was carried out, involving two groups of voluntary student participants. As the target audience for this research, all participating students were on courses studying for Level 3 qualifications, including colleges and university Foundation years. Both groups engaged with an activity in a game-based learning environment to solve a problem involving the use of computational thinking concepts and practices. To ensure a strict standard of quality, both groups:

- Attempted to solve the same problem.
- Spent a similar amount of time in the activity.
- Had participants decided by random volunteer sampling.

The **control group** (A) attempted to solve the problem with no modification to the environment or learning process, whereas the **variant group** (B) experienced a modified version of the learning environment that incorporated the gameful design principle of implicit progression, intended to interface with and support the learning process.

After the activity, participants from both groups answered a questionnaire asking them to provide a series of self-assessments on the CT concepts they used in solving the problem. These self-assessments pertained to the confidence they have in identifying and assessing their CT skills, the level to which they rate their proficiency in the application of each of the CT concepts, and the level of interest the participants had in continuing to learn and improve their CT skills through similar problem-solving activities.

1) Research Philosophy: The philosophical position of this research is primarily **Inferential**, involving generalized conclusions deduced through evidence and reasoning. Data from the Level 3 student participants has been used to make inferences and predictions about students studying Level 3 qualifications as a whole, with consideration for potential participant bias or limitations in the research.

Due to the involvement with running the activity and potential collaboration between students, this research also inspires from **Interpretivism**, in which it is acknowledged that different students may require differing levels of assistance to effectively engage with the research. To ensure that all students had a fair chance at engaging with the research, cross-collaboration and flexibility in aiding participants with the problem-solving process was allowed. The limitations to this assistance include:

1) Solutions were not directly given. Only components and methods allowing for a solution were explained.
2) CT concepts required for solutions were not identified. Only their required functions were explained.

Though this approach is likely to have lead to some bias in the results, these measures are predicted to have minimised it, and ensured the benefits of allowing all participants to effectively engage with problem-solving through flexible support far outweigh any potential bias from researcher intervention.

2) Playful Learning Environment: Influenced by the GBL research reviewed within the Contextual Analysis, as well as personal experience and proficiency with the environment, 'Minecraft' [60] was the chosen playful learning environment for this research, and effectively behaves as the user interface for the modification and problem-solving activity.

This choice was also influenced by the popularity of Minecraft within the likely age group of the participants, with reports stating that a majority 43% of its player base to be between the ages 15-21 [116]. It was therefore predicted that a number of participants would have a positive association and existing experience with playing the game, minimising the risk of usability problems with the rules and mechanics of the learning environment obstructing research.

The game offers a variety of 'blocks' that players can freely use to build in the sandbox setting of its virtual world. This research focusses on the usage of 'Redstone' blocks [117], which allow for the simulation of computational logic within the game, with upward possible complexity of working computers [118]. For the activity, participants were required to use Redstone blocks to simulate the logic of the following CT concepts (henceforth referred to as 'components'):

1) **AND Gate** [119], [120], outputting a signal only when provided two simultaneous input signals.
2) **NOT Gate** [120], [121], outputting a signal when no input signal is provided.
3) **RS Latch** [122], [123], storing an 'off' (RESET) or 'on' (SET) state based on the last input signal given.
4) **Clock Generator** [124], [125], outputting a repeated signal of consistent frequency. In the context of the activity, it is used with a conditional to simulate a loop.

Additionally, steps were taken to exclude the irrelevant mechanics and blocks of Minecraft from the test, with a pre-prepared 'world' containing the problem for participants to solve, as well as providing the participants with all blocks required to solve the problem. Additional function was added to integrate the modification into the problem-solving process in the variant test, allowing participants to test individual components of the implemented solution.

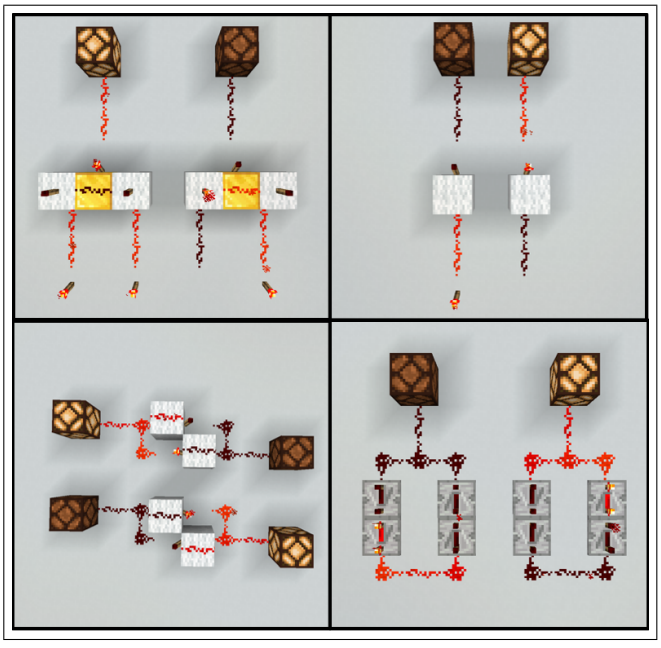


Figure 5. Implementations of AND Gates (top-left), NOT Gates (top-right), RS Latches (bottom-left) and Clock Generators (bottom-right) in Minecraft.

3) The Problem to be Solved: The problem for the participants to solve was designed to balance complexity and approachability, such that it challenges students to exercise and consequently improve their CT skills to solve it, but does not discourage engagement. Following this, involving practices of decomposition, pattern recognition, abstraction and the development of algorithms in its solution was a priority.

Another consideration for its design was encouraging a playful mindset for problem-solving, allowing for a reduced fear of failure, a greater openness to challenge and the stimulation of intrinsic motivation. To address this, the problem was designed to be a minigame that the participants could play upon a successfully implemented solution.

Figure 6. Interface for the minigame, with four input colours and a win/lose display along the top. The purple-coloured pillars represent broken components of the minigame requiring fixing.

To satisfy all aforementioned requirements, the minigame to be implemented was modelled after the mechanics of the Simon memory game [126] to support recognisability and a straightforward understanding of solution requirements, with the mechanics of the game simplified to a single sequence of five random colours, outputting a 'Win' on a successful matching input, and 'Lose' if the input colours are incorrect.

The minigame logic was built by the author to exclusively use simple boolean logic of AND, OR and NOT logic gates, boolean memory storage and a single use of iteration, with the exception of boolean randomisers unintended for participant interaction. After creation, thirteen instances of simple components were removed from the minigame and highlighted with purple colours for participant implementation, including:

- Eight AND Gates
- Three RS Latches
- One NOT Gate
- One Clock Generator

The highlighted removal of required components had the aim of increasing the ease of participant engagement and clarity of problem-solving, as well as encourage the use of CT practices (decomposition, pattern recognition and abstraction) to understand, use, or otherwise disregard the characteristics of the wider solution to implement its specific requirements, allowing for multiple problem-solving approaches. The minigame is notably comprised of four tracks of five repeated 'modules' used to generate, store and match the 'true' and 'false' values of two colours, enabling participants to use other parts of the solution (such as a working 'module') as a reference and guide for problem-solving. The overall function of the minigame is further outlined in Appendix A.

4) Computing Artefact: The computing artefact developed for this research was a modification for Minecraft, developed using the **Java** [127] programming language. Developing this modification involved the following software:

- **Java Development Kit** (JDK) [128], to write programs, compile code and build the JAR file for the mod.
- **IntellJ IDEA** [129], an integrated development environment (IDE) used to create and manipulate Java class files.
- **Forge Mod Developer Kit** (MDK) [130], to make the mod compatible with the Forge mod loader [131], which simplifies compatibility between Minecraft and mods.
- **Paint.NET** [132], for creating custom game sprites.

This artefact was developed subject to an **Agile** [133] development cycle, involving the concurrent design, prototyping and testing of the modification's aspects based on priority and immediate requirements, with flexibility for those requirements to change as knowledge was gathered.

The first aspect to be developed was an in-game method to test the logic implemented by participants, with the goal of ensuring if an individual component was correct during the process of active problem-solving, and subsequently enabling a progressional reward associated with the implemented CT concept. This reflected a need for in-game **unit testing** [134], and consequently lead to the following requirements:

- **Input** - The implemented component needed to be provided a series of inputs that could identify if the component was correct dependent on how they were processed.
  1) Different components needed different input sequences to confirm their function.
  2) Components requiring two sources of input (AND Gate, RS Latch) needed both input sources to provide different input sequences.
  3) Components requiring two sources of input needed both input sources to start together and be 'in-time'.
- **Output** - The outputs of the implemented component needed measuring to confirm if the component had the correct output for every given input.
  1) The measurement of output was to only begin once testing input sources had started.
  2) The output of implemented components needed to be compared against the intended output of what the component was supposed to be.
  3) A correct sequence of outputs for the intended component needed to result in confirmation to the user that the component is correct, along with identification of the implemented CT concept.
  4) An incorrect sequence of outputs for the intended component needed to result in a message to the user acknowledging that the component is incorrect.
- **Isolation** - Component testing needed to prevent surrounding logic from interfering with the test until component confirmation or rejection.
  1) The stored values of RS Latches connected to the component needed to be either temporarily disconnected, or otherwise set in correspondance with the intended component inputs.

An **Input Block** was developed to address the first requirement, with the functionality to be placed in the world and assigned a specific sequence to output once activated through a received signal, allowing participants to interact with an in-game button to trigger a testing sequence. For components requiring multiple input sources, these button inputs simply triggered two Input Blocks with separate assigned sequences simultaneously. The sequences of Input Blocks included:

- Input - **AND Gate 1** - 0, 0, 1, 1
- Input - **AND Gate 2** - 0, 1, 0, 1
- Input - **NOT Gate** - 0, 1
- Input - **RS Latch 1** - 1, 0, 0, 0
- Input - **RS Latch 2** - 0, 0, 1, 0
- Input - **Clock Generator** - 1

Following this, an **Output Block** was developed to address the secondary requirement, with similar functionality to be placed in the world and assigned a sequence to detect. To ensure that Output Blocks only began testing when both Input Blocks were enabled, Input Blocks were programmed to output a signal to their corresponding Output Block when activated, only after which the Output Block begins checking for an input sequence. The checked sequences of Output Blocks included:

- Output - **AND Gate** - 0, 0, 0, 1
- Output - **NOT Gate** - 1, 0
- Output - **RS Latch 1** - 1, 1, 0, 0
- Output - **RS Latch 2** - 0, 0, 1, 1
- Output - **Clock Generator** - 0, 1, 1

Figure 7. An example of a successfully implemented AND Gate, involving user feedback and the reward of an AND Gate block (edited with increased sizes for readability)

Notably, Output Blocks were required to test two input sequences for the two states of an RS Latch, in which the storage of input signals without currently active input, as well as the disabling of one state when the other state is activated, was to result in a correct implementation. It's also worth drawing attention to a correctly implemented Clock Generator only requiring a single input signal to subsequently result in multiple output signals, in which the Output Block was designed to test if the frequency of inputs was perfectly consistent to confirm a successful implementation. The detection of other components (AND Gate, NOT Gate) were simply resultant from the immediate state of input at the intervals an Output Block tested for the next input(s) in the sequence.

Lastly, the method through which component tests were isolated exclusively involved in-game construction and use of Redstone mechanics, with situational application to specific components where needed. Not all components needing implementation required complete isolation, however those that did had differing requirements dependent on surrounding logic.

All methods of testing, including the use of Input Blocks, Output Blocks and measures to isolate components, were incorporated to be hidden from participants, with the only source of interaction involving accessible buttons on pink blocks used to activate component tests.

After the successful implementation of in-game blocks for unit-testing implemented components, the next required feature of the modification was the development of progressional rewards embedded within the process of problem-solving, with the intent for these implicit rewards to reflect and explicitly identify the CT concepts successfully implemented by participants without interrupting the 'flow' of learning. To this extent, the aim was to define these rewards as 'tools' that seamlessly and implicitly integrate with the process of problem-solving.

Following this, the implementation of the required CT concepts as usable blocks fulfilled those requirements nicely, in which the blocks carry out the function of a multi-block component within the space of a single block. This allows the successful implementation of any successive component to only require identification of what CT concept the component is, and subsequent placement of the corresponding block. It follows that the blocks to be implemented included:

1) **AND Gate Block** - Takes in inputs from two sides, outputting a signal forwards if provided two input signals.
2) **NOT Gate Block** - Takes in an input from three sides, outputting a signal forwards if no signal is provided.
3) **RS Latch** - Takes in inputs from two sides, storing and outputting the last given signal to its source side.
4) **Clock Generator** - Outputs a limited number of consistent-frequency signals when provided a signal.

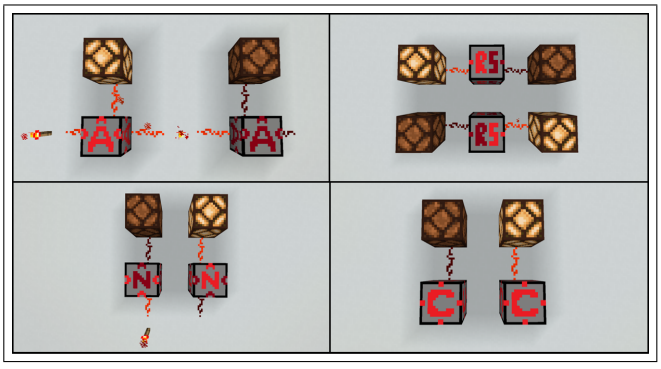


Figure 8. Tool blocks, including AND Gates (top-left), NOT Gates (top-right), RS Latches (bottom-left) and Clock Generators(bottom-right).

After the tool blocks were implemented, the 'advancement' system of Minecraft was leveraged to award tool blocks to players upon successfully implementing their corresponding CT concepts. This summarised the feature implementation of the modification, with all instances of component testing and tool blocks verified to work in all required contexts, further outlined in the Artefact Testing section in Appendix C.

For code refactoring, the required functions of most tool blocks (AND Gate, NOT Gate, RS Latch) were noticed to shared many similarities to the existing 'Diode' block type in Minecraft, with the 'DiodeBlock' class having many useful methods for receiving input signals and processing those signals into output. As such, classes for most tool blocks were developed through extending the 'DiodeBlock' class and overriding a number of its methods for the specific functions required, with the exception of the Clock Generator block.

In terms of the Input, Output and Clock Generator blocks, their respective requirements of altering behaviour dependent on the passage of game time and saving/loading data, such as assigned sequences and timings, meant they needed implementation as 'Block Entities'. As such, their main functions were performed in classes extending from the 'BlockEntity' class, with key methods such as 'tick', 'load' and 'saveAdditional' being overridden to carry out the specific behaviours required.

After implementation, a 'TickingEntity' interface along with extended 'SequenceBlock' and 'SequenceEntity' classes were created for their shared behaviour. A UML diagram for the majority of classes can be found in Appendix B.

5) Questionnaire Design: With intention to statistically verify or reject the hypotheses in Figure I, quantitative data was collected from both groups in a questionnaire after the activity, subject to the following questions:

1) **On a scale of 1-10, how confident are you to rate your understanding of X?**
   - Logic gates (AND, OR NOT)
   - Data storage constructs (SR latches, Flip-flops)
   - Clock generators and similar forms of iteration
2) **If you can, rate on a scale of 1-10 your skill level in the application of X to solving complex problems.** (Optional).
   - Logic gates
   - Data storage constructs
   - Iteration
3) **If you answered the prior question, how confident do you feel in your rating? (1-5).**
4) **On a scale of 1-10, how interested are you in further problem solving involving the aforementioned algorithmic constructs?**.

Questions 1-3 were asked for each generalised CT concept relating to the required components of the activity, whereas Question 4 was asked solely at the end of the questionnaire. The answers given by students were then combined, attributed to each hypothesis in the following format:

H1: All Question 1 and 3 type answers, raised to the same numerical significance.
H2: All Question 2 type answers.
H3: All Question 4 answers.

These collections of quantitative data were then analysed to confirm or reject their corresponding hypotheses, subject to the Statistical Testing process outlined below.

Github Repository:
https://github.falmouth.ac.uk/GA-Undergrad-Student-Work-24-25/COMP303-2200066-CT-Skills-Self-perception-Dissertation.git

6) Statistical Testing: For each of the hypotheses, an independent-samples t-test was conducted using the corresponding collections of data for each of the groups, in which data collections involving multiple columns were averaged into a single column of one data point per participant. These tests were performed with a significance level of 5% and a beta probability of 0.8, with an estimated effect size of 0.8 (large) for all hypotheses. All tests were one-tailed, intending to measure a significant increase in the variant data.

Due to limitations in ascertaining research participants (further described in the Discussion section), the achieved sample size for group A was a total of 26 participants, whereas the achieved sample size for group B was a total of 11 participants. An analysis using G*Power [135] thus finds an overall power of 0.704 (3 d.p), resulting in an approximate 30% probability of error. For an ideal minimum power of 0.8, a secondary analysis shows a minimum sample size of 21 participants per group to be required for statistical significance.

Nonetheless, tests were performed for each hypothesis with the goal of evaluating whether the results of the **variant group** (B) were significantly higher than the corresponding results of the **control group** (A). The code for these statistical tests, performed in RStudio [136], are outlined in Appendix E.

VI. Ethical Considerations

As this research involves human participants, it adheres to the ethical values of the Nuremberg Code [137] and the Helsinki Declaration [138], including informed consent ensured through voluntary sampling and signed agreement. The study was held in the educational institutions of the participants and involved supervision throughout, ensuring the research was safe with participants able to leave at any point. The circumstances and goals of the research were also clearly explained to participants through a slide deck before their participation, with a mandatory and informed prompt for consent before questionnaire data could be submitted.

The study ensures the protection of personal data of the participants, complying with the UK General Data Protection Regulation (UK GDPR) [139] and the Data Protection Act 2018 [140]. This study also complies with the Research Ethics and Integrity Policy of Falmouth University [141].

The study and developed computing artefact complies with the usage guidelines, user agreements and licenses of 'Minecraft' [142] and all development software used [143] [144], as well the behavioural guidelines of the schools and educational institutions that this study took place in. The artefact will not be used in any commercial context.

Sustainability concerns include the use of multiple computers, corresponding to the sample size, over the duration of multiple hour-long sessions. However, the energy use of this is considered to be minimal and similar to the existing daily activities of the educational institutions involved in the research, thus the overall impact is considered to be low.

Overall, the ethical risk of this study is estimated to be medium risk, in which care has been taken to positively represent Falmouth University, the academic institution of this research, and align with the British Computer Society (BCS) Code of Conduct [145].

VII. Data Management

In correspondence with the aforementioned legal frameworks [139], [140], all questionnaire data has been collected and securely stored through Microsoft Forms with permission from participants and the involved educational institutions. The data is accessible through the author's university account associated with the email provided by this paper.

For the purpose of optional data withdrawal, first names of the participants have been stored with informed consent, with the author's email provided to the participants and colleges to allow data exclusion at any point. As outlined in the Questionnaire Design section, the Quantitive questionnaire data of each participant group (A/B) has been combined into three data collections corresponding to the research hypotheses, with temporarily storage on the author's local PC across six separate CSV files for processing in three independent-samples t-tests, as outlined in the Statistical Testing section.

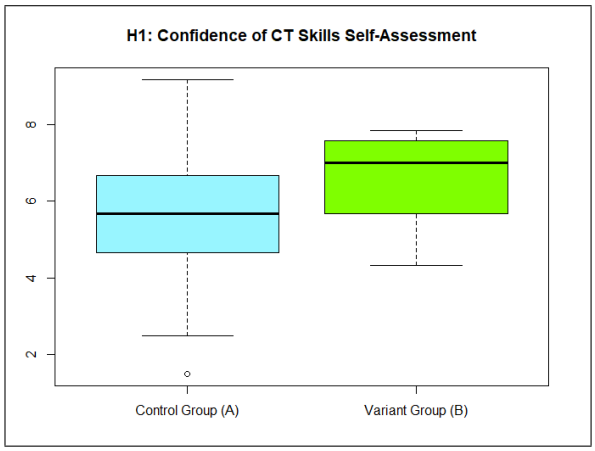
All data has been analysed using RStudio [136], subject to the variables laid out in the Statistical Testing section and using the R code shown in Appendix E. After processing, the data has been withheld until the submission of this paper in case further processing is required. There will be a maximum retention date of 26 May 2025, after which it will be deleted. Questionnaire data is the only data that has been collected from participants.

VIII. Results and Analysis

Three sessions of data collection occurred over a two-week period. This amounted to two sessions for the unmodified Control Test for a total of **26 participants** in the control group (A), and one session for the modified Variant Test for a total of **11 participants** in the variant group (B).

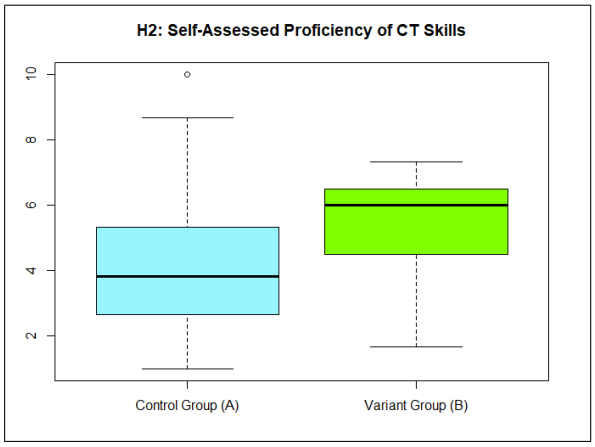
A. Hypothesis 1 - Confidence of Self-assessment

The t-test for the confidence students had in assessing their CT skills resulted in a p-value of $p = 0.0467$ and a t-value of $t = 1.7401$, with sample means of 5.628 (A) and 6.545 (B) (3 d.p). As the p-value meets the significance level (5%), the data leans towards rejecting the null hypothesis, however the low power of the test and small degree to which the p-value is significant means this data will require further discussion.



B. Hypothesis 2 - Self-assessed rating of Proficiency

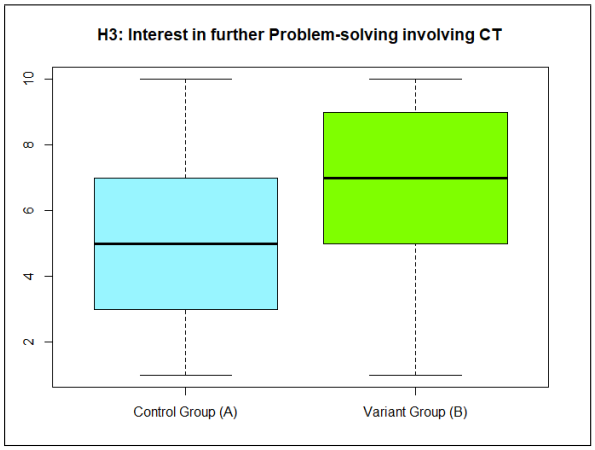
The t-test for the rating of proficiency students had in self-assessing their CT skills resulted in a p-value of $p = 0.03956$ and a t-value of $t = 1.8244$, with sample means of 4.243 (A) and 5.454 (B) (3 d.p). Though the p-value of this test is lower than that of Hypothesis 1, displaying a likelier rejection of the null hypothesis, the risk of a statistical error from low-power means these results will also require further discussion.



A notable pattern shared between this data and the data of the first hypothesis is an overall higher variance of the control group (A) compared to an overall lower variance of the variant group (B). The variant group (B) also shows a consistent negative skew, in contrast to the more symmetrical distribution of the control group (A), albeit with the presence of outliers. The small sample size of the variant group (B) is likely to have an impact on this skew, however further exploration is needed to better understand the reasons for these patterns.

C. Hypothesis 3 - Interest in further Problem-solving

The t-test for the interest students had in further Problem-solving using CT skills resulted in a p-value of $p = 0.09887$ and a t-value of $t = 1.3422$, with sample means of 5.23 (A) and 6.545 (B) (3 d.p). This data shows insufficient evidence to reject the null hypothesis, as well as an abnormally high variation in the results for both groups. This is examined in the following section.



IX. Discussion

Altogether, without consideration for statistical error or limitations, the results present enough evidence to reject the null hypotheses of Hypothesis 1 and Hypothesis 2, with students displaying a significantly increased level of confidence and self-efficacy in their self-assessments from the presence of implicit progression within the playful learning environment, however there is insufficient evidence to reject the null hypothesis of Hypothesis 3, with no proven support for whether student interest in future problem-solving is increased by incorporating implicit progression into the environment.

A. Implications of Findings

The similarity of the results for Hypothesis 1 and Hypothesis 2 in terms of the p-value and t-value of the overall results, as well as the consistent distributions of each group, shows a likely correlation between the confidence of self-assessment and assessed proficiency with CT skills. This outlines that students who were more confident to assess their CT skills also had a higher level of self-efficacy in those skills. Whether there is a degree of causation between the two (self-efficacy increasing confidence of self-assessment or confidence of self-assessment increasing self-efficacy) cannot be determined, although it is not unlikely for both to increase proportionally.

The overall lower range and strong negative skew of the variant group (B) compared to the control group (A) may also point to an enhanced self-perception leading to a generally more optimistic self-assessment with a lower likelihood of extreme highs or lows, resulting in a greater degree of consistency across a population, similarly supported by the lack of outliers compared to those in the control group (A).

Overall, the wider range of the results for the control group (A), alongside the lower means for both Hypothesis 1 and Hypothesis 2, points to an overall lower ability to formulate a self-perception of implicitly learnt CT skills compared to the students in the variant group (B), who's practised skills were explicitly identified and encapsulated as usable 'tools'. To this extent, the data supports the idea that the inclusion of implicit progression helped to build a stronger connection between the practice-based learning and application of CT skills and the subsequent effect on self-perception of those skills.

In terms of the results for Hypothesis 3, the lack of sufficient evidence to reject the null hypothesis can be observed to primarily derive from the high degree of variation, in which the difference in sample means is otherwise more significant than those in Hypothesis 1. In terms of why this variation exists for both groups, it is most likely that participant interest in further problem-solving is modulated by too many factors unconsidered by the research for the inclusion of implicit progression to significantly affect the patterns of response.

B. Research Limitations

Firstly, the low statistical power (0.704) of the test was a severe limitation to the validity of the results, with an approximate 30% chance of error. This is significantly lower than the ideal minimum power of 0.8, with the high chance of statistical error meaning further research is needed before the null hypotheses can be confidently rejected.

This limitation was aligned with the difficulty of attaining an ideal minimum sample size of 21 participants for each group. This was due to the unanticipated difficulty with installing the required software for the Variant Test on college computers, including a Forge [131] version of Minecraft and a corresponding version of Java [128] to be installed for each participant, in which a variety of potential educational institutions for the research were unable to fulfil this due to technical restrictions. Ultimately, the Variant Test was carried out with university Foundation years, satisfying the requirement for students studying a Level 3 qualification whilst alleviating the technical restrictions. In contrast, the unmodified Control Test was able to use Minecraft Education [61], which was accessible at the college the research took place at, resulting in a disproportionally higher number of participants in the Control Test compared to the Variant Test.

An additional limitation resulting from this is the potential imbalance between the groups, in which all participants of the control group (A) were college students (ages 16-18), whereas all participants of the variant group (B) were Foundation year university students of varying ages. As ages were not recorded, it cannot be accurately assessed how this affected the results, however it is likely that the increased confidence and self-efficacy of the self-assessments of the variant groups' Foundation year students compared to the control groups' college students is correlated with a higher level of age, experience and existing CT skills. The general unpredictability of prior experience and existing CT skills may also be correlated with large variations of participant results within each group, as can be seen in the results of the control group (A).

Additionally, all participants were on courses related to games and game development due to the difficulty of engaging students without an existing interest in the subject of the research. Although this is common to both groups, ensuring little to no skew of the results, it may draw attention to the results being only relevant to students studying in Level 3 games courses, as opposed to the wider range of students studying Level 3 qualifications as a whole. The influence of individual differences such as gender, race, background and learning differences were also not considered, for which further research would be required to understand the link between this research and different student demographics.

Another significant limitation for this research was found in the utilised GBL environment of Minecraft [60], in which the internal Redstone [117] mechanics of the game, though enabling the implementation of computational logic, is subject to the unique and isolated rules of the game. Consequently, the rules of how to use Redstone components, as well as the overall rules of Minecraft, needed to be understood by participants for effective creation of computational logic. Although an introductory presentation was used to provide all necessary information about Redstone mechanics, the time it took participants to practice the use of Redstone mechanics obstructed the implemention of CT concepts, with some participants struggling more with the internal mechanics of Minecraft than the actual problem-solving activity.

This introduced further bias, with some participants having significantly more experience with using Minecraft than others, including strong familiarity with the implementation of in-game computational logic. These students were often able to complete the activity with greater speed and success than others, potentially enhancing their self-perception beyond students less familiar with Minecraft, even if sharing previous similarities in CT skills self-perception.

This limitation was also echoed in Y.Pan's paper [110] involving Minecraft, which recommended for future research to provide more tutorial levels to help students become familiar with fundamental game operations, with the interest of maximising learning engagement on subject matter instead of game rules and actions. This illustrates that the limitation may be inherent to the environment, in which further measures could have been used to minimise its effect. Notably however, all participants had existing familiarity with Minecraft and were able to play the game with no serious issues, such that the activity was never entirely obstructed for any student.

X. Conclusion

Altogether, though the results of Hypothesis 1 and Hypothesis 2 show enough statistical significance to reject their null hypotheses, this study concludes there to be insufficient evidence to confidently reject any of the null hypotheses with regards to the low power of the statistical tests and limitations from imbalanced participant groups. However, the results showed promise that with a greater sample size and thus higher-powered test, it is likely that a meaningful connection between gameful design principles and the enhancement of student self-perceptions of CT skills could be found.

Consequently, further research is required to prove whether incorporating the gameful design principle of implicit progression into a playful learning environment significantly enhances the self-perception of CT skills used within the environment, with recommendations for similar studies to include:

1) Careful choice of the playful learning environment to be used, with consideration for advantages and disadvantages respective to the research requirements.
2) Thorough measures of onboarding and support to enable all participants an equal chance at engaging with the challenges of problem-solving without obstruction.
3) Development of methods other than participant self-assessment for measuring self-perception of skills.
4) Deeper exploration of other principles of gameful design and their effects on student self-perception of skills.

A. Future Work

In terms of further research in this area of study, a deeper and more specific analysis into game-based learning environments could allow for a better understanding of how one could be modified for enhanced player self-perception. Additionally, a focus on the role of creativity and player freedom within playful problem-solving activities may provide significant insight into the relation between the principles of gameful design and playful learning as a whole.

REFERENCES

[1] Mabel Addis. The Sumerian Game. (1964). IBM. [video game].

[2] Don Rawitsch, Bill Heinemann, Paul Dillenberger. The Oregon Trail. (1971). [video game]. Available: https://oregontrail.ws/ (Accessed: 12 December 2024).

[3] Adipat, S. et al. (2021) 'Engaging students in the learning process with game-based learning: The fundamental concepts', International Journal of Technology in Education, 4(3), pp. 542–552. doi:10.46328/ijte.169.

[4] Imperial College London, "How can I engage through games and play?" Available at: https://www.imperial.ac.uk/media/imperial-college/be-inspired/societal-engagement/public/How-do-I-engage-through-games-and-play.pdf (Accessed: 12 December 2024).

[5] Fulya Eyupoglu, T. and Nietfeld, J.L. (2019) 'Intrinsic motivation in game-based Learning Environments', Advances in Game-Based Learning, pp. 85–102. doi:10.1007/978-3-030-15569-8_5.

[6] Gamer motivation model (2016) Quantic Foundry. Available at: https://quanticfoundry.com/gamer-motivation-model/ (Accessed: 12 December 2024).

[7] Implicit Learning - an overview — ScienceDirect Topics. Available at: https://www.sciencedirect.com/topics/neuroscience/implicit-learning (Accessed: 12 December 2024).

[8] Resnick, M. (2003) Playful learning and creative societies - MIT media lab, Playful Learning and Creative Societies. Available at: https://web.media.mit.edu/ mres/papers/education-update.pdf (Accessed: 12 December 2024).

[9] Rice, L. (2009) 'Playful learning', Journal for Education in the Built Environment, 4(2), pp. 94–108. doi:10.11120/jebe.2009.04020094.

[10] Blinkoff, E. et al. (2023) 'Investigating the contributions of active, playful learning to student interest and educational outcomes', Acta Psychologica, 238, p. 103983. doi:10.1016/j.actpsy.2023.103983.

[11] What is computational thinking? - introduction to computational thinking - KS3 computer science revision - BBC bitesize (2023) BBC News. Available at: https://www.bbc.co.uk/bitesize/guides/zp92mp3/revision/1 (Accessed: 12 December 2024).

[12] Lee, J., Joswick, C. and Pole, K. (2022) 'Classroom play and activities to support computational thinking development in early childhood', Early Childhood Education Journal, 51(3), pp. 457–468. doi:10.1007/s10643-022-01319-0.

[13] Stephen Wolfram. (2016) "How to Teach Computational Thinking—Stephen Wolfram Writings," Stephenwolfram.com. Available at: https://writings.stephenwolfram.com/2016/09/how-to-teach-computational-thinking/ (Accessed: 12 December 2024).

[14] Resnick, M. et al. (2009) Scratch: Programming for all: Communications of the ACM: Vol 52, no 11, Communications of the ACM. Available at: https://dl.acm.org/doi/10.1145/1592761.1592779 (Accessed: 17 December 2024).

[15] Broza, O., Biberman-Shalev, L. and Chamo, N. (2023) '"Start from scratch": Integrating computational thinking skills in teacher education program', Thinking Skills and Creativity, 48, p. 101285. doi:10.1016/j.tsc.2023.101285.

[16] E. Team, "What is GBL (game-based learning)?," EdTechReview, https://www.edtechreview.in/dictionary/what-is-game-based-learning/ (Accessed: Dec. 18, 2024).

[17] "Learning games: Make learning awesome!," Kahoot!, https://kahoot.com/ (Accessed: Dec. 19, 2024).

[18] "Learn a language for free," Duolingo, https://www.duolingo.com/ (Accessed: Dec. 19, 2024).

[19] J. Matulef, "Kerbal Space Program lands on various schools' curriculum," Eurogamer.net, https://www.eurogamer.net/kerbal-space-program-lands-on-various-schools-curriculum (accessed Apr. 14, 2025).

[20] K. Doss, V. Juarez, D. Vincent, P. Doerschuk, and J. Liu, "Work in progress — A survey of popular game creation platforms used for computing education," 2011 Frontiers in Education Conference (FIE), Oct. 2011. doi:10.1109/fie.2011.6143110

[21] G. Byun, J. Moon, and C. Sun, "Enhancing computational thinking through constructionist gaming in a roblox-supported Virtual Makerspace," Practitioner Proceedings of the 10th International Conference of the Immersive Learning Research Network (iLRN2024), pp. 32–35, 2024. doi:10.56198/5m1rhngtm

[22] "Kodu Game Lab," KoduGameLab, https://www.kodugamelab.com/ (accessed Apr. 14, 2025).

[23] Meerbaum-Salant, O., Armoni, M. and Ben-Ari, M. (Moti) (2013) 'Learning computer science concepts with scratch', Computer Science Education, 23(3), pp. 239–264. doi:10.1080/08993408.2013.832022.

[24] "The state of the field of computational thinking in early childhood education," OECD Education Working Papers, Jul. 2022. doi:10.1787/3354387a-en

[25] S. Yeni, J. Nijenhuis-Voogt, M. Saeli, E. Barendsen, and F. Hermans, "Computational thinking integrated in school subjects – a cross-case analysis of students' experiences," International Journal of Child-Computer Interaction, vol. 42, p. 100696, Dec. 2024. doi:10.1016/j.ijcci.2024.100696

[26] T. PALTS and M. PEDASTE, "A model for developing Computational Thinking Skills," Informatics in Education, vol. 19, no. 1, pp. 113–128, Mar. 2020. doi:10.15388/infedu.2020.06

[27] Stadler, M.A. (1997) 'Distinguishing implicit and explicit learning', Psychonomic Bulletin & Review, 4(1), pp. 56–62. doi:10.3758/bf03210774.

[28] Chevalier, A. et al. (2007) 'Students' academic self-perception', SSRN Electronic Journal [Preprint]. doi:10.2139/ssrn.1017507.

[29] Ghazvini, S.D. (2011) 'Relationships between academic self-concept and academic performance in high school students', Procedia - Social and Behavioral Sciences, 15, pp. 1034–1039. doi:10.1016/j.sbspro.2011.03.235.

[30] Tirri, K. and Nokelainen, P. (2010) 'The influence of self-perception of abilities and attribution styles on academic choices: Implications for gifted education', Roeper Review, 33(1), pp. 26–32. doi:10.1080/02783193.2011.530204.

[31] Johnston, O., Wildy, H. and Shand, J. (2022) 'Teenagers learn through play too: Communicating high expectations through a playful learning approach', The Australian Educational Researcher, 50(3), pp. 921–940. doi:10.1007/s13384-022-00534-3.

[32] The scientific case for learning through play. Available at: https://learningthroughplay.com/explore-the-research/the-scientific-case-for-learning-through-play (Accessed: 17 December 2024).

[33] Next gen. nesta. Available at: https://www.nesta.org.uk/report/next-gen/ (Accessed: 17 December 2024).

[34] "Computing education: Royal Society," The Royal Society, Available at: https://royalsociety.org/news-resources/projects/computing-education/ (Accessed: Dec. 18, 2024).

[35] Brown, N.C. et al. (2013) 'Bringing computer science back into schools', Proceeding of the 44th ACM technical symposium on Computer science education, pp. 269–274. doi:10.1145/2445196.2445277.

[36] A. Bandura, "Self-efficacy: Toward a unifying theory of behavioral change.," Psychological Review, vol. 84, no. 2, pp. 191–215, 1977. doi:10.1037//0033-295x.84.2.191

[37] 'Engaging and re-engaging students in learning at school' (2008) PsycEXTRA Dataset [Preprint]. doi:10.1037/e574462011-001.

[38] M. Csikszentmihalyi, Flow: The Psychology of Optimal Experience. New York: Simon & Schuster, 1994.

[39] F. Rheinberg, "Intrinsic motivation and flow.," Motivation Science, vol. 6, no. 3, pp. 199–200, Sep. 2020. doi:10.1037/mot0000165

[40] J. Nakamura, D. C. K. Tse, and S. Shankland, "Flow," The Oxford Handbook of Human Motivation, pp. 167–185, Aug. 2019. doi:10.1093/oxfordhb/9780190666453.013.10

[41] J. Vervaeke, L. Ferraro, and A. Herrera-Bennett, "Flow as spontaneous thought," Oxford Handbooks Online, Apr. 2018. doi:10.1093/oxfordhb/9780190464745.013.8

[42] C. Dichev, D. Dicheva, G. Angelova, and G. Agre, "From gamification to gameful design and gameful experience in learning," Cybernetics and Information Technologies, vol. 14, no. 4, pp. 80–100, Jan. 2015. doi:10.1515/cait-2014-0007

[43] G. F. Tondello, A. Mora, and L. E. Nacke, "Elements of gameful design emerging from user preferences," Proceedings of the Annual Symposium on Computer-Human Interaction in Play, pp. 129–142, Oct. 2017. doi:10.1145/3116595.3116627

[44] H. G. Group, "Elements of gameful design classified by user preferences," Medium, https://medium.com/gameful-design/elements-of-gameful-design-classified-by-user-preferences-61b21cbdd435 (Accessed: Dec. 17, 2024).

[45] M. K. Kim, "Models of learning progress in solving complex problems: Expertise development in teaching and learning," Contemporary Educational Psychology, vol. 42, pp. 1–16, Jul. 2015. doi:10.1016/j.cedpsych.2015.03.005

[46] Garvey, C. (1977) Play.

[47] B. Conrad, Constructivism, May 2022. doi:10.4324/9781138609877-ree32-1

[48] N. Whitton and A. Moseley, Playful Learning: Events and Activities to Engage Adults. New York, NY: Routledge, 2019.

[49] N. Whitton, "A manifesto for playful learning," Play and Learning in Adulthood, pp. 87–124, 2022. doi:10.1007/978-3-031-13975-8_4

[50] R. T. Nørgård, C. Toft-Nielsen, and N. Whitton, "Playful learning in higher education: Developing a signature pedagogy," International Journal of Play, vol. 6, no. 3, pp. 272–282, Sep. 2017. doi:10.1080/21594937.2017.1382997

[51] N. Whitton, "Playful learning: Tools, techniques, and Tactics," Research in Learning Technology, vol. 26, no. 0, May 2018. doi:10.25304/rlt.v26.2035

[52] H. Borge Garnaas and R. van den Tillaar, "Implicit versus explicit learning a novel skill for high school students," Journal of Motor Behavior, vol. 56, no. 6, pp. 697–704, Jul. 2024. doi:10.1080/00222895.2024.2375553

[53] J. Cohn, P. Squire, I. Estabrooke, and E. O'Neill, "Enhancing intuitive decision making through implicit learning," Lecture Notes in Computer Science, pp. 401–409, 2013. doi:10.1007/978-3-642-39454-6_42

[54] J. McMullin, "Using design games," Boxes and Arrows, https://boxesandarrows.com/using-design-games/ (Accessed: Dec. 19, 2024).

[55] M. Salmond, Video Game Design: Principles and Practices from the Ground Up. Bloomsbury Academic, 2020.

[56] R. Zubek, Elements of Game Design. Cambridge, MA: The MIT Press, 2020.

[57] K. Salen and E. Zimmerman, Rules of Play: Game Design Fundamentals. Cambridge, Mass: The MIT Press, 2010.

[58] O. Alawajee and J. Delafield-Butt, "Minecraft in education benefits learning and social engagement," International Journal of Game-Based Learning, vol. 11, no. 4, pp. 19–56, Oct. 2021. doi:10.4018/ijgbl.2021100102

[59] E. J. Slattery, D. Butler, M. O'Leary, and K. Marshall, "Teachers' experiences of using minecraft education in primary school: An Irish perspective," Irish Educational Studies, vol. 43, no. 4, pp. 965–984, Mar. 2023. doi:10.1080/03323315.2023.2185276

[60] "Welcome to the official site of Minecraft," Minecraft.net, https://www.minecraft.net/en-us (Accessed: Dec. 19, 2024).

[61] "Minecraft Education," education.minecraft.net, https://education.minecraft.net/en-us (Accessed: Dec. 19, 2024).

[62] R. Brooks, "What is computational thinking?," University of York, https://online.york.ac.uk/what-is-computational-thinking/ (Accessed: Dec. 19, 2024).

[63] J. Wing, Computational thinking, https://www.cs.cmu.edu/ 15110-s13/Wing06-ct.pdf (Accessed: Dec. 19, 2024).

[64] T.-C. Hsu, S.-C. Chang, and Y.-T. Hung, "How to learn and how to teach computational thinking: Suggestions based on a review of the literature," Computers & Education, vol. 126, pp. 296–310, Nov. 2018. doi:10.1016/j.compedu.2018.07.004

[65] D. F. Wood, "ABC of Learning and teaching in medicine: Problem based learning," BMJ, vol. 326, no. 7384, pp. 328–330, Feb. 2003. doi:10.1136/bmj.326.7384.328

[66] R. Gillies, "Cooperative learning: Review of research and practice," Australian Journal of Teacher Education, vol. 41, no. 3, pp. 39–54, Mar. 2016. doi:10.14221/ajte.2016v41n3.3

[67] E. H. J. Yew and K. Goh, "Problem-based learning: An overview of its process and impact on learning," Health Professions Education, vol. 2, no. 2, pp. 75–79, Dec. 2016. doi:10.1016/j.hpe.2016.01.004

[68] "Problem based learning: A facilitator of computational thinking," Proceedings of the 18th European Conference on e-Learning, p. 36, Dec. 2019. doi:10.34190/eel.19.150

[69] R. W. Jensen and C. C. Tonies, Software Engineering. Englewood Cliffs, N.J: Prentice-Hall, 1979.

[70] C. C. Foster and T. Iberall, Computer Architecture. New York, N.Y: Van Nostrand Reinhold Co, 1985.

[71] I. N. Šerbec, (2023). The Overview of Computer Science/Computational Thinking Education Research.

[72] M. S. Kadir and A. S. Yeung, "Academic self-concept," Encyclopedia of Personality and Individual Differences, pp. 9–16, 2020. doi:10.1007/978-3-319-24612-3_1118

[73] G. C. Nobre and N. C. Valentini, "Autopercepção de Competência em Crianças: Conceito, Mudanças Características na Infância E fatores associados," Journal of Physical Education, vol. 30, no. 1, p. 3008, Dec. 2018. doi:10.4025/jphyseduc.v30i1.3008

[74] D. J. Bem, "Self-perception: An alternative interpretation of cognitive dissonance phenomena.," Psychological Review, vol. 74, no. 3, pp. 183–200, 1967. doi:10.1037/h0024835

[75] B. Simonsmeier, H. Peiffer, M. Flaig, and M. Schneider, "Peer feedback improves students' academic self-concept in higher education," Research in Higher Education, vol. 61, no. 6, pp. 706–724, Mar. 2020. doi:10.1007/s11162-020-09591-y

[76] A. S. Yeung, R. G. Craven, and G. Kaur, "Mastery goal, value and self-concept: What do they predict?," Educational Research, vol. 54, no. 4, pp. 469–482, Nov. 2012. doi:10.1080/00131881.2012.734728

[77] J. S. Mathew, "Self-perception and academic achievement," Indian Journal of Science and Technology, vol. 10, no. 14, pp. 1–6, Apr. 2017. doi:10.17485/ijst/2017/v10i14/107586

[78] H. W. Marsh and A. J. Martin, "Academic self-concept and academic achievement: Relations and causal ordering," British Journal of Educational Psychology, vol. 81, no. 1, pp. 59–77, Mar. 2011. doi:10.1348/000709910x503501

[79] S. Kulakow, "Academic self-concept and achievement motivation among adolescent students in different learning environments: Does competence-support matter?," Learning and Motivation, vol. 70, p. 101632, May 2020. doi:10.1016/j.lmot.2020.101632

[80] S. Deterding, D. Dixon, R. Khaled, and L. Nacke, "From game design elements to gamefulness," Proceedings of the 15th International Academic MindTrek Conference: Envisioning Future Media Environments, pp. 9–15, Sep. 2011. doi:10.1145/2181037.2181040

[81] R. N. Landers et al., "Defining gameful experience as a psychological state caused by gameplay: Replacing the term 'gamefulness' with three distinct constructs," International Journal of Human-Computer Studies, vol. 127, pp. 81–94, Jul. 2019. doi:10.1016/j.ijhcs.2018.08.003

[82] Kenny, "Gameful design vs Gamification vs Serious Games," Gametize Hub, https://corp.gametize.com/2022/04/21/post-gameful-design-vs-gamification-vs-serious-games/ (Accessed: Dec. 20, 2024).

[83] R. Mitchell, L. Schuster, and H. S. Jin, "Gamification and the impact of extrinsic motivation on needs satisfaction: Making work fun?," Journal of Business Research, vol. 106, pp. 323–330, Jan. 2020. doi:10.1016/j.jbusres.2018.11.022

[84] M. Jackson, Gamification elements to use for learning, https://trainingindustry.com/content/uploads/2017/07/enspire_cs_gamification_2016.pdf (Accessed: Dec. 20, 2024).

[85] S. Deterding, "The lens of intrinsic skill atoms: A method for gameful design," Human–Computer Interaction, vol. 30, no. 3–4, pp. 294–335, May 2015. doi:10.1080/07370024.2014.993471

[86] R. M. Ryan and E. L. Deci, "Self-determination theory and the facilitation of intrinsic motivation, social development, and well-being.," American Psychologist, vol. 55, no. 1, pp. 68–78, 2000. doi:10.1037//0003-066x.55.1.68

[87] A. Toxboe, "Self-determination theory," Learning Loop, https://learningloop.io/glossary/self-determination-theory-sdt (Accessed: Dec. 20, 2024).

[88] R. M. Ryan and M. Vansteenkiste, "Self-determination theory," The Oxford Handbook of Self-Determination Theory, pp. 3–30, Feb. 2023. doi:10.1093/oxfordhb/9780197600047.013.2

[89] A. Brazie, "Game progression and progression systems," Game Design Skills, https://gamedesignskills.com/game-design/game-progression/ (Accessed: Dec. 20, 2024).

[90] B. Strååt and H. Verhagen, "Exploring video game design and player retention- a longitudinal case study," Proceedings of the 22nd International Academic Mindtrek Conference, pp. 39–48, Oct. 2018. doi:10.1145/3275116.3275140

[91] D. Maranhão, A. Rocha Franco, and J. Maia, (PDF) Assessing Emergence and Progression in games, https://www.researchgate.net/publication/315737783_Assessing_Emergence_and_Progression_in_Games (Accessed: Dec. 20, 2024).

[92] Ellis, R. (2009) '1. implicit and explicit learning, knowledge and instruction', Implicit and Explicit Knowledge in Second Language Learning, Testing and Teaching, pp. 3–26. doi:10.21832/9781847691767-003.

[93] A. Cleeremans, "Principles for implicit learning," How Implicit Is Implicit Learning?, pp. 195–234, Oct. 1997. doi:10.1093/acprof:oso/9780198523512.003.0008

[94] A. Cleeremans, "Implicit learning and implicit memory," Encyclopedia of Consciousness, pp. 369–381, 2009. doi:10.1016/b778-012373873-8.00047-5

[95] A. S. Reber, Implicit learning and Tacit Knowledge, Sep. 1996. doi:10.1093/acprof:oso/9780195106589.001.0001

[96] "Conscious versus unconscious cognition," The Nature of Cognition, pp. 173–204, Feb. 1999. doi:10.7551/mitpress/4877.003.0009

[97] H. Van Keer and J. P. Verhaeghe, "Effects of explicit reading strategies instruction and peer tutoring on second and fifth graders' reading comprehension and self-efficacy perceptions," The Journal of Experimental Education, vol. 73, no. 4, pp. 291–329, Jul. 2005. doi:10.3200/jexe.73.4.291-329

[98] M. Boroumand, M. Mardani, and F. Khakzad Esfahlan, "The influence of explicit teaching and utilization of concept mapping on EFL

students' listening comprehension and perceived self-efficacy," Language Teaching Research Quarterly, vol. 21, pp. 16–34, Jan. 2021. doi:10.32038/ltrq.2021.21.02

[99] T. Koponen et al., "Benefits of integrating an explicit self-efficacy intervention with calculation strategy training for low-performing elementary students," Frontiers in Psychology, vol. 12, Aug. 2021. doi:10.3389/fpsyg.2021.714379

[100] I. Govender et al., "Increasing self-efficacy in learning to program: Exploring the benefits of explicit instruction for problem solving," The Journal for Transdisciplinary Research in Southern Africa, vol. 10, no. 1, Jul. 2014. doi:10.4102/td.v10i1.19

[101] L. Leifheit, "The role of self-concept and motivation within the 'computational thinking' approach to Early Computer Science Education," Publikationsserver der Universität Tübingen, https://publikationen.uni-tuebingen.de/xmlui/handle/10900/113938 (Accessed: Dec. 19, 2024).

[102] L. Leifheit et al., "Development of a questionnaire on self-concept, motivational beliefs, and attitude towards programming," Proceedings of the 14th Workshop in Primary and Secondary Computing Education, pp. 1–9, Oct. 2019. doi:10.1145/3361721.3361730

[103] T. Ollis, "A critical pedagogy of Embodied Education," A Critical Pedagogy of Embodied Education, pp. 209–225, 2012. doi:10.1057/9781137016447_8

[104] B. R. Belland, "Computer-based scaffolding strategy," Instructional Scaffolding in STEM Education, pp. 107–126, Oct. 2016. doi:10.1007/978-3-319-02565-0_5

[105] C. R. Graham, "Blended learning," Education, Aug. 2016. doi:10.1093/obo/9780199756810-0156

[106] P. MIHCI Türker and F. K. Pala, "The effect of algorithm education on students' computer programming self-efficacy perceptions and computational thinking skills," International Journal of Computer Science Education in Schools, vol. 3, no. 3, pp. 19–32, Jan. 2020. doi:10.21585/ijcses.v3i3.69

[107] F. B. Tek, K. S. Benli and E. Deveci, "Implicit Theories and Self-Efficacy in an Introductory Programming Course," in IEEE Transactions on Education, vol. 61, no. 3, pp. 218-225, Aug. 2018, doi: 10.1109/TE.2017.2789183.

[108] C.-H. Wu, Y.-C. Chien, M.-T. Chou, and Y.-M. Huang, "Integrating computational thinking, Game Design, and design thinking: A scoping review on trends, applications, and implications for Education," Humanities and Social Sciences Communications, vol. 12, no. 1, Feb. 2025. doi:10.1057/s41599-025-04502-x

[109] R. Razzouk and V. Shute, "What is design thinking and why is it important?," Review of Educational Research, vol. 82, no. 3, pp. 330–348, Sep. 2012. doi:10.3102/0034654312457429

[110] Y. Pan, E. L. Adams, L. R. Ketterlin-Geller, E. C. Larson, and C. Clark, "Enhancing middle school students' computational thinking competency through game-based learning," Educational technology research and development, vol. 72, no. 6, pp. 3391–3419, Jul. 2024. doi:10.1007/s11423-024-10400-x

[111] E. Kutay and D. Oner, "Coding with Minecraft: The development of Middle School Students' computational thinking," ACM Transactions on Computing Education, vol. 22, no. 2, pp. 1–19, Feb. 2022. doi:10.1145/3471573

[112] E. J. Slattery, P. Lehane, D. Butler, M. O'Leary, and K. Marshall, "Assessing the benefits of digital game-based learning with minecraft in children, adolescents and young adults: A broad systematic review," Review of Education, vol. 13, no. 1, Jan. 2025. doi:10.1002/rev3.70035

[113] C. Hébert and J. Jenson, "Teaching with sandbox games: Minecraft, game-based learning, and 21st century competencies," Canadian Journal of Learning and Technology, vol. 46, no. 3, May 2021. doi:10.21432/cjlt27990

[114] M. Nkadimeng and P. Ankiewicz, "The affordances of Minecraft Education as a game-based learning tool for atomic structure in Junior High School Science Education," Journal of Science Education and Technology, vol. 31, no. 5, pp. 605–620, Jul. 2022. doi:10.1007/s10956-022-09981-0

[115] Q. Liu, "Minecraft redstone: The gateway to engineering," Lecture Notes in Education Psychology and Public Media, vol. 59, no. 1, Jul. 2024. doi:10.54254/2753-7048/59/20241766

[116] "Minecraft user statistics: How many people play minecraft in 2024?," SearchLogistics, https://www.searchlogistics.com/learn/statistics/minecraft-user-statistics/ (Accessed: Dec. 19, 2024).

[117] "Redstone components," Minecraft Wiki, https://minecraft.fandom.com/wiki/Redstone_components (Accessed: Dec. 20, 2024).

[118] "Tutorials/Redstone Computers," Minecraft Wiki, https://minecraft.fandom.com/wiki/Tutorials/Redstone_computers (Accessed: Dec. 19, 2024).

[119] "And gates - boolean logic - OCR - GCSE computer science revision - OCR - BBC Bitesize," BBC News, https://www.bbc.co.uk/bitesize/guides/zjw8jty/revision/2 (accessed Apr. 13, 2025).

[120] "Redstone circuits/logic," Minecraft Wiki, https://minecraft.fandom.com/wiki/Redstone_circuits/Logic (accessed Apr. 13, 2025).

[121] "Or gates - boolean logic - OCR - GCSE computer science revision - OCR - BBC Bitesize," BBC News, https://www.bbc.co.uk/bitesize/guides/zjw8jty/revision/3 (accessed Apr. 13, 2025).

[122] O. M. Urias, "The S-R latch (quickstart tutorial)," Build Electronic Circuits, https://www.build-electronic-circuits.com/s-r-latch/ (accessed Apr. 13, 2025).

[123] "Redstone circuits/memory," Minecraft Wiki, https://minecraft.fandom.com/wiki/Redstone_circuits/Memory (accessed Apr. 13, 2025).

[124] "Iteration - iteration in programming - KS3 computer science revision - BBC bitesize," BBC News, https://www.bbc.co.uk/bitesize/guides/z3khpv4/revision/1 (accessed Apr. 13, 2025).

[125] "Redstone circuits/clock," Minecraft Wiki, https://minecraft.fandom.com/wiki/Redstone_circuits/Clock (accessed Apr. 13, 2025).

[126] "Hasbro games Simon Game For Kids - Hasbro," Hasbro Instructions, https://instructions.hasbro.com/en-us/instruction/simon-game-for-kids-ages-8-and-up (Accessed: Dec. 19, 2024).

[127] Java, https://www.java.com/en/ (accessed Apr. 14, 2025).

[128] "Download the latest Java Lts Free," Oracle United Kingdom, https://www.oracle.com/uk/java/technologies/downloads/

[129] JetBrains, "IntelliJ IDEA – the leading Java and Kotlin Ide," JetBrains, https://www.jetbrains.com/idea/ (Accessed: Dec. 19, 2024).

[130] "Getting started with forge," Introduction - Forge Documentation, https://docs.minecraftforge.net/en/latest/gettingstarted/

[131] "Minecraft forge downloads," Forge Files, https://files.minecraftforge.net/net/minecraftforge/forge/index_1.20.1.html (accessed Apr. 14, 2025).

[132] R. Brewster, "Paint.net - free software for digital photo editing," PaintNET RSS, https://www.getpaint.net/ (Accessed: Dec. 19, 2024).

[133] J. Foley, "What is Agile Software Development?," Agile Alliance —, https://www.agilealliance.org/agile101/ (Accessed: Dec. 19, 2024).

[134] "What is unit testing?," smartbear.com, https://smartbear.com/learn/automated-testing/what-is-unit-testing/

[135] "G*Power," Psychologie, https://www.psychologie.hhu.de/arbeitsgruppen/allgemeine-psychologie-und-arbeitspsychologie/gpower (Accessed: Dec. 20, 2024).

[136] "RStudio desktop," Posit, https://posit.co/download/rstudio-desktop/#download (Accessed: Dec. 20, 2024).

[137] "Nuremberg Code," UNC Research, https://research.unc.edu/human-research-ethics/resources/ccm3_019064/ (Accessed: Dec. 20, 2024).

[138] "WMA - The World Medical Association-WMA Declaration of helsinki – ethical principles for medical research involving human participants," The World Medical Association, https://www.wma.net/policies-post/wma-declaration-of-helsinki/ (Accessed: Dec. 20, 2024).

[139] Regulation (EU) 2016/679 of the European Parliament and of the Council, https://www.legislation.gov.uk/eur/2016/679/contents (Accessed: Dec. 20, 2024).

[140] "Data protection act 2018," Legislation.gov.uk, https://www.legislation.gov.uk/ukpga/2018/12/contents (Accessed: Dec. 20, 2024).

[141] "Research and knowledge exchange integrity and ethics policy," Falmouth University, https://www.falmouth.ac.uk/sites/default/files/media/downloads/Research Ethics_Integrity_Policy_(1)_0.pdf (Accessed: Dec. 20, 2024).

[142] "Usage guidelines," Minecraft.net, https://www.minecraft.net/en-us/usage-guidelines (Accessed: Dec. 20, 2024).

[143] "JetBrains User Agreement," JetBrains, https://www.jetbrains.com/legal/docs/toolbox/user/

[144] R. Brewster, "Licensing and FAQ," Paint.NET, https://www.getpaint.net/license.html (Accessed: Dec. 20, 2024).

[145] "BCS Code of conduct," BCS, https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/ (Accessed: Dec. 20, 2024).

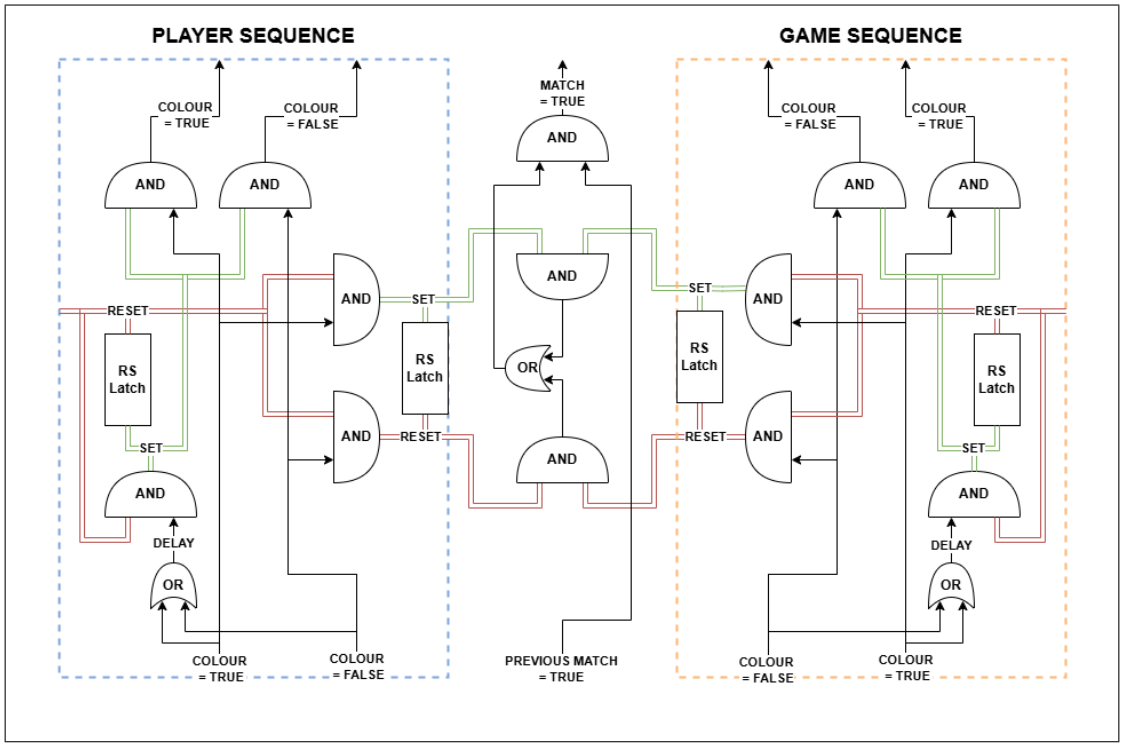


Figure 9. The player/game sequences takes a colour input of 'True' or 'False', and either stores the input for comparison against the other, or passes the signal along to the next module in the sequence if already set. If this and all previous modules in the sequence successfully match, it outputs a signal.

Appendix A
Minigame Logic

The diagram shown in Figure 9 illustrates the primary logic of the minigame participants attempted to implement, in which the four colours of the minigame are each associated with a sequence of five of these logical modules, with each module representing a value in the sequence. Each module is comprised of a 'Player sequence' side and a 'Game sequence' side, with each side storing an inputted 'True' or 'False' value for a colour. If both sides are storing the same value, the module outputs a signal for a successful match, but only if the previous module is inputting a signal for a successful match.

To store a value, the module must be in the 'RESET' state, and will subsequently enter a 'SET' state after successfully storing a value. If the module is in the 'SET' state when receiving an input, it will instead pass the input to the next module in the sequence. Once all modules in the sequence are 'SET', the sequence is finished and a full colour match can be determined, compared with every other colour and consequently output a 'Win' or 'Lose' depending on the result.

By incorporating these sequences with input from the player or game, wherein each colour input correlates to a 'False' input for every other colour, the minigame is able to store a sequence of five 'True' or 'False' values for each input colour and subsequently compare all sequences to see if the player input and game input match, allowing the memory game to work correctly. This overall design also ensures that the reimplementation of removed components always results in a working solution, as the logic is only dependent on correct input/output processing without regard for any other factors.

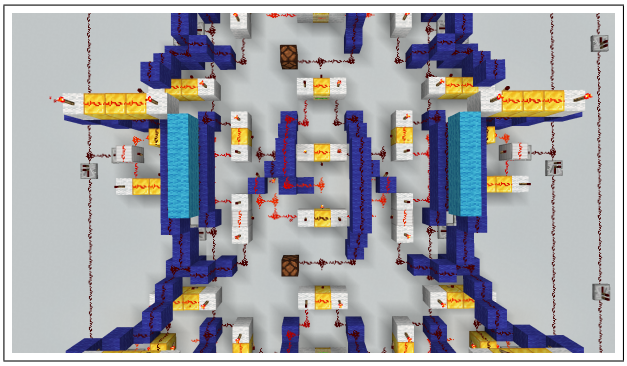


Figure 10. An example of one such module

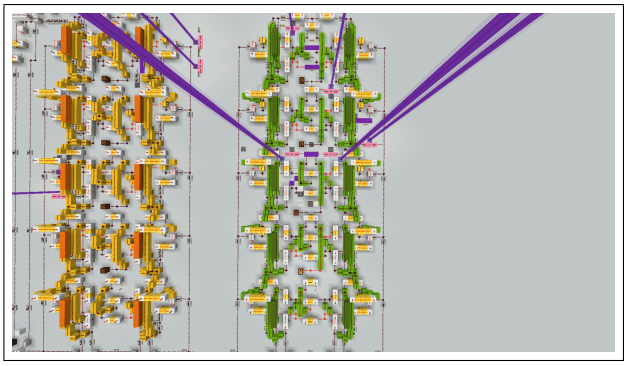
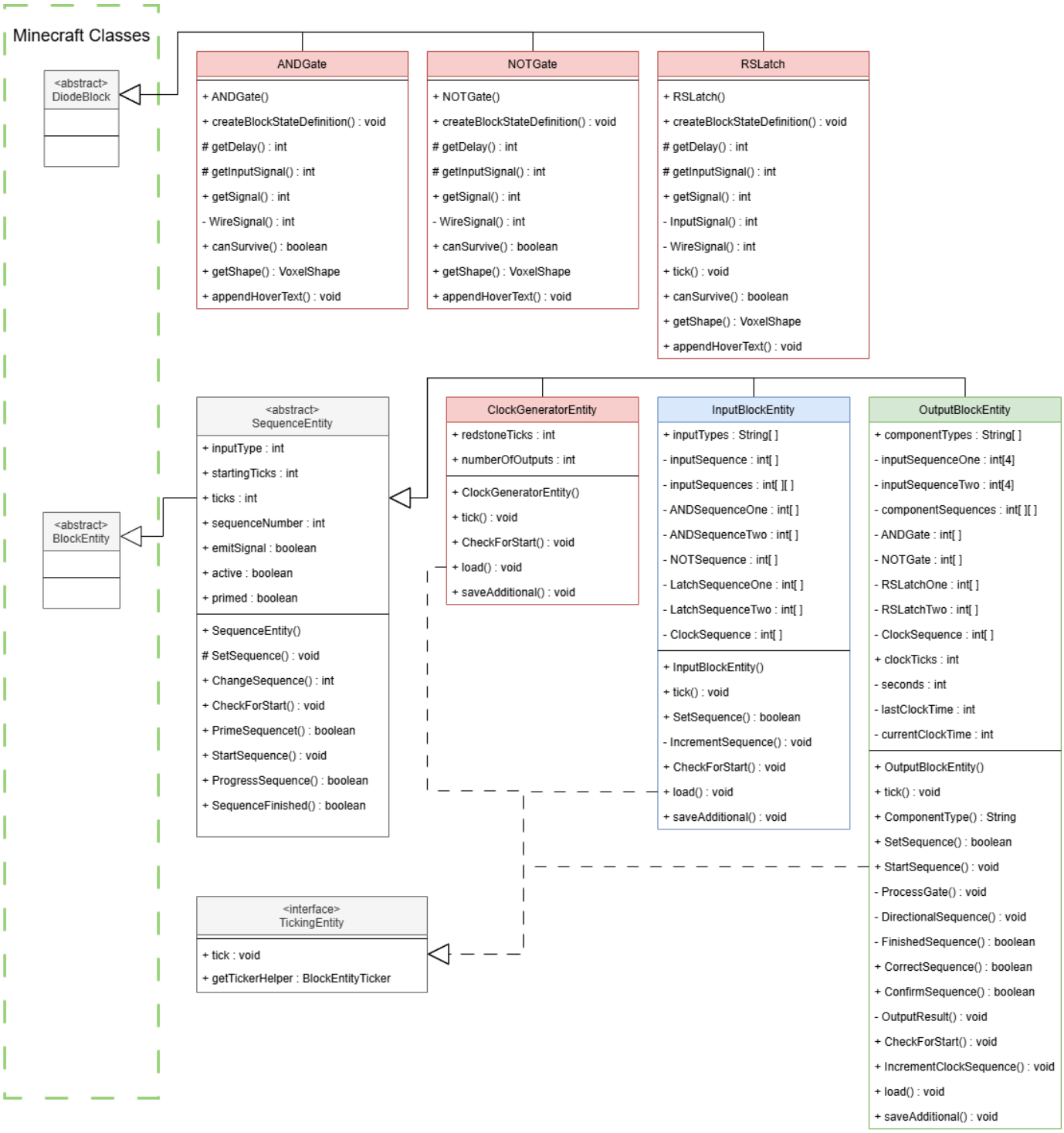


Figure 11. One of the four colour tracks, with five modules in sequence. Some components are removed for participant implementation.

Appendix B
UML Class Diagram



APPENDIX C
ARTEFACT TESTING

*1) **Agile Development Cycle**:* Development of the artefact took place over a series of four 1-week sprints, in which the end of each sprint involved a planned deliverable. These deliverables were planned with flexible adaption to the immediate requirements of the modification, as well as available time. The sprint timeline of the deliverables were as follows:

1) Design and creation of the problem-solving activity
2) **Input Block** and **Output Block** functionality
3) Integrating component testing and progressional rewards into the activity using **Input Blocks** and **Output Blocks**
4) Rewarded Tool Blocks, including the **AND Gate**, **NOT Gate**, **RS Latch** and **Clock Generator** blocks

After each sprint, the deliverables were tested and refined with regards to the other methods outlined in the testing process.

*2) **Activity Tests**:* At the end of every sprint, acceptance testing of the problem-solving activity was conducted. This was initially performed without the modification until its integration in Sprint 3, after which it was carried out with both the modified and unmodified environments. This included:

- Testing the win/lose cases of the **implemented** minigame.
- Removing the components intended for participant implementation and running the **unimplemented** minigame.
- **Reimplementing** the components, running the minigame after each one until the win/lose cases work.
- (**Modified**) Replacing reimplemented components with corresponding Tool Blocks, testing win/lose cases.
- (**Modified**) Running all component tests after each component implementation and removal, ensuring incorrect/-correct components are properly detected.

*3) **Unit Tests**:* A difficulty presented for code testing was the heavy reliance on the Minecraft environment for most methods. However, a few key unit tests were developed for the decision-making behaviour of Input Blocks and Output Blocks, contained within the 'SequenceEntityTest' and 'OutputBlockEntityTest' classes. Additionally, due to their behaviour as unit tests for in-game components, these blocks were subsequently used as unit tests for all Tool Blocks after they were assured to work through unit and acceptance testing.

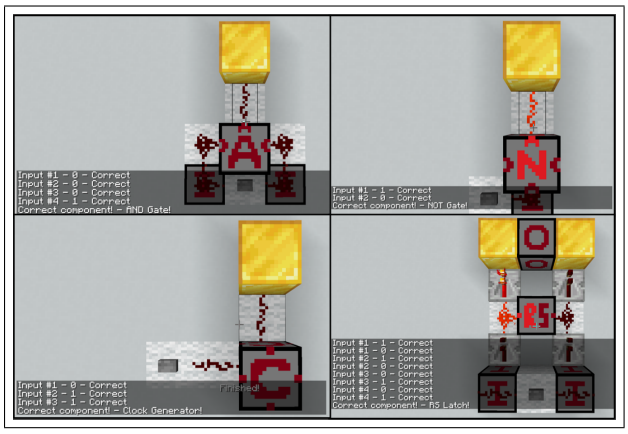


Figure 12. Successful in-game unit tests of all Tool Block functions through the use of Input Blocks and Output Blocks.

*4) **User Testing**:* In addition to the Activity Tests conducted by the author, an outside user lacking experience with CT skills was involved with testing the activity after each sprint to ensure it was achievable, with provided informal feedback relating to usability and enjoyment. This feedback was then utilised to streamline the process of the activity, removing any glaring obstructions or sources of confusion.

*5) **Build Testing**:* Lastly, once the core implementation of the modification was completed, it was built into a single .jar file and installed on the matching version of Minecraft Forge (1.20.1 - 47.0.19), after which all previously described tests were repeated to guarantee a fully-functional problem-solving activity with a correctly interfacing modification.

APPENDIX D
REFLECTIVE ADDENDUM

A. Activity Design

When designing the activity, I had mostly followed my intuition of what would be an interesting and fun problem to solve, with no deeper research or plan. With hindsight, the minigame can be observed to have glaring issues of space constraints and readability, in which every component to be implemented needed to be specific or else it wouldn't fit. This was massively detrimental to the intended openness for creativity, and was poorly addressed by the allowance for participants to simply copy components from similar areas.

Furthermore, a primary goal of the activity was for participants to deduce the intended function by decomposing the surrounding game logic. However, the logic was simply too dense, resultant from unexpected complexities during creation. In-game signs used to explain intended functions relative to the minigame were often useless because of this, and should have been replaced with isolated explanations of input/output.

These issues were caused by a lack of consistent user testing, as well creating the activity from start to finish without peer feedback, by which point it was too late to redesign it. To improve upon this, my goal is to work more closely and frequently with a peer during the creation of future participant activities, to ensure a higher quality user experience.

B. Artefact Functionality

The function of the modification was significantly delayed, resulting in the available time window for the Variant Test to be extremely small and delaying the knowledge of technical requirements, inhibiting prior communications with colleges.

To this extent, the lack of a timely artefact significantly impacted the success of the study, in which more time to organise research and work through technical limitations could have allowed for a larger variant group (B) sample size, and therefore a greater statistical power for a confident conclusion. To address these issues, I plan to strongly prioritise the development of uncertain and decisive aspects of future research.

C. Conclusion

In conclusion, though this study was beset by my failings of planning, timing and individualist approach to development, I am nonetheless proud of the software and results produced, and will take the learned hindsight into my future projects.

APPENDIX E
STATISTICAL TEST CODE (R)

```
library(readr)

# H1 Data - Generate a single column of means
ControlDataH1 <- rowMeans(read_csv("ControlDataH1.csv"))
VariantDataH1 <- rowMeans(read_csv("VariantDataH1.csv"))

boxplot(ControlDataH1, VariantDataH1,
        main = "H1: Confidence of CT Skills Self-Assessment",
        names = c("Control Group", "Variant Group"),
        col = c("cadetblue1", "chartreuse1"))

## p-value > 0.05 - Null: No significant increase in confidence of assessment
## p-value < 0.05 - Alt: Significant increase in confidence of assessment
# Independent two-sample, One-sided
t.test(VariantDataH1, ControlDataH1, "greater", mu=0, FALSE, FALSE, 0.95)

# H2 Data - Generate a single column of means
ControlDataH2 <- rowMeans(read_csv("ControlDataH2.csv"))
VariantDataH2 <- rowMeans(read_csv("VariantDataH2.csv"))

boxplot(ControlDataH2, VariantDataH2,
        main = "H2: Self-Assessed Proficiency of CT Skills",
        names = c("Control Group", "Variant Group"),
        col = c("cadetblue1", "chartreuse1"))

## p-value > 0.05 - Null: No significant increase in rating of proficiency
## p-value < 0.05 - Alt: Significant increase in rating of proficiency
# Independent two-sample, One-sided
t.test(VariantDataH2, ControlDataH2, "greater", mu=0, FALSE, FALSE, 0.95)

# H3 Data
ControlDataH3 <- rowMeans(read_csv("ControlDataH3.csv"))
VariantDataH3 <- rowMeans(read_csv("VariantDataH3.csv"))

boxplot(ControlDataH3, VariantDataH3,
        main = "H3: Interest in further Problem-solving involving CT",
        names = c("Control Group", "Variant Group"),
        col = c("cadetblue1", "chartreuse1"))

## p-value > 0.05 - Null: No significant increase of interest in further problem-solving
## p-value < 0.05 - Alt: Significantly increased interest in further problem-solving
# Independent two-sample, One-sided
t.test(VariantDataH3, ControlDataH3, "greater", mu=0, FALSE, FALSE, 0.95)
```